



Inheritance and Object-Oriented Principles

Practical Skills Module 4: Topic 2

[Visit our website](#)

Practical Skills Module 4: Topic 2

Topic Code	PM-04-PS02
Topic	Use inheritance, polymorphism, encapsulation and abstraction

Scope of Practical Skill

Given a range the learner must be able to:

- PA0201 Define a class
- PA0202 Create an object of the class
- PA0203 Use inheritance to create a class which uses all the properties and behaviour of another class
- PA0204 Implement polymorphism in various ways
- PA0205 Use encapsulation to restrict access to methods and variables and prevent data from direct modification
- PA0206 Use abstraction to hide internal details and show only functionalities

Applied Knowledge

- AK0201 Concept, definition and functions of intermediate Python programming principles

Internal Assessment Criteria

- IAC0201 Expected results with Python OOP inheritance, polymorphism, encapsulation and abstraction are achieved

Resources:

Knowledge	Information from Python Programmer Curriculum
National Curriculum Framework	900221-000-00-PM-04



Practical task

Follow the facilitator's instructions to complete the following activities:

Scenario: Library Management System

A library wants to create a **Python program** to manage books, staff, and members. The system will use **inheritance**, **polymorphism**, **encapsulation**, and **abstraction** to efficiently organize and secure functionalities.

Create a Python file called **library_management_system.py** and complete the steps below:

Step 1: Define a Class

Create a base class called **Person** to represent common attributes and behaviours shared by people in the library system (e.g., staff and members).

Include:

- A constructor (`__init__`) with attributes such as:
 - `name` (person's full name)
 - `person_id` (a unique identifier)
- A method to display the person's basic details.

Expected input example (values used when creating an object):

```
Name: "Sam Taylor"
Person ID: "P001"
```

Step 2: Create an Object of the Class

Create an object of the `Person` class using sample data to represent a library user. Use the object to call the method that displays the person's details.

Expected output example:

```
Name: Sam Taylor  
ID: P001
```

Step 3: Use Inheritance to Create a Class That Reuses Another Class

Create two subclasses:

- `Staff`
- `Member`

Each subclass should:

- Inherit from `Person`
- Add at least one unique attribute:
 - For `Staff`, this could be `role` (e.g., "Librarian")
 - For `Member`, this could be `membership_type` (e.g., "Standard" or "Premium")

Ensure these classes can still use properties from the `Person` class.

Step 4: Implement Polymorphism in Various Ways

- Override the method used to display details in both `Staff` and `Member` classes. Each overridden method should display the basic details and the subclass-specific information.

Expected output example:

```
Name: Sam Taylor
```

```
ID: P001
Role: Librarian
```

```
Name: Jane Doe
ID: P002
Membership Type: Premium
```

- Create a function (outside the classes) that takes a `Person` object as a parameter and calls the `display` method.
Use it with both `Staff` and `Member` objects to demonstrate polymorphism.

Step 5: Use Encapsulation to Restrict Access

Implement **encapsulation** in the `Person` class:

- Make at least one attribute private (e.g., `__person_id`)
- Provide **getter** and **setter** methods to access or update this attribute safely.

Demonstrate that direct access to the private attribute is restricted.

Step 6: Use Abstraction to Hide Internal Details

Use the `abc` module to:

- Make `Person` an **abstract class**
- Include at least one **abstract method** (e.g., `display_details()`)
- Ensure `Staff` and `Member` provide their own implementation of this method



Take note

Important: Be sure to upload all files required for the task submission inside your task folder and then click "Request review" on your dashboard.
